Slide 1 / 16

YOLOv8 Object Detection with ONNX and RKNN

Target recording time: 00:00 to 00:57

Recording guide: approximately 119 words per minute, with a 60-second demonstration on slide 10. Read narration only. Cues and source lines are not spoken. Rehearse once to adjust pauses. Narration matches the PowerPoint speaker notes.

Hello everyone. In this presentation, we will discuss our group project on object detection using YOLOv8, ONNX, and Rockchip's RKNN Toolkit2. Our goal was to understand how a pretrained model moves through a deployment workflow and produces a useful detection image.

We had to set up the ONNX components and the RKNN toolkit, prepare the model, and run the conversion scripts in the correct order. We then examined the saved images to see how the detector identified objects in a street scene. We will explain the technologies first, walk through the setup and conversion, show our results, and finish with our group's reflections. The presentation also includes a short screen demonstration of the conversion workflow.

Slide 2 / 16

Project scope

Target recording time: 00:57 to 01:58

The scope of this project was deployment and experimentation with an existing object detector. We started with the nano version of YOLOv8, rather than training a new neural network from the beginning. This allowed us to focus on the practical work required to make the model usable in another software environment.

Our main deliverables were a working conversion workflow, an exported RKNN model, and images showing object detection. The repository also includes several other inference backends, but our discussion concentrates on the ONNX and RKNN path. We will distinguish the evidence available in our saved files from measurements we would need for a larger evaluation. This matters because completing model conversion and measuring performance on an edge board are separate milestones.

Sources (not spoken): Local PROJECT_DISCUSSION.md, INSTRUCTIONS.md, README.md

YOLOv8 and object detection

Target recording time: 01:58 to 03:02

Object detection answers two questions about an image: what objects appear, and where are they located? For each selected detection, the model provides information that our software turns into a category label, a bounding box, and a confidence score. Multiple objects can appear in the same image.

YOLOv8 is the detector used in this project. Ultralytics describes its detection architecture as using an anchor-free head. Our scripts select the pretrained nano model, called YOLOv8n. They contain eighty category labels, including person, bus, bicycle, and car.

The visible boxes in our final result come from this detection pipeline. The model supplies predictions, and the surrounding Python code filters and draws them. Understanding that division helped us see why correct postprocessing is necessary even after the model runs successfully.

Sources (not spoken): https://docs.ultralytics.com/models/yolov8/ ; local inference scripts

ONNX and ONNX Runtime

Target recording time: 03:02 to 04:05

ONNX stands for Open Neural Network Exchange. It provides a representation of a model's computational graph, including its operators, inputs, outputs, and parameters. In our workflow, that representation connects the original PyTorch-based model with the RKNN conversion tool.

ONNX Runtime has a different responsibility. It executes a compatible model using an inference session. This distinction explains why we needed to set up both ONNX and ONNX Runtime. One supports the model representation, while the other provides an execution path.

The repository includes an ONNX Runtime inference script, which can run detection on the host computer. That path is useful when checking the input and output handling independently of the RKNN conversion sequence. It also provides a possible baseline for future comparisons with the converted model.

Sources (not spoken): https://onnx.ai/onnx/intro/ ; https://onnxruntime.ai/docs/ ; local onnxruntime_inference.py

Slide 5 / 16

RKNN and the RK3588 target

Target recording time: 04:05 to 05:10

RKNN Toolkit2 handles the Rockchip part of our workflow. Its Python interface lets us load the ONNX model, configure the target platform, perform conversion, and export an RKNN model. Our conversion scripts import the RKNN class from the rknn dot api module.

The separate board inference script uses RKNN Lite. Rockchip describes Toolkit-Lite2 as providing Python interfaces for deployment on supported Rockchip hardware. The assignment connects this software stack with the RK3588 and the Orange Pi 5 Plus deployment context.

Our scripts set the conversion target to RK3588. This tells the converter which platform to prepare for. However, that setting alone does not establish where a particular saved image was generated. Board execution needs its own runtime configuration and evidence, such as a recorded run on the device.

Sources (not spoken): https://github.com/airockchip/rknn-toolkit2 ; local rknn_lite_inference.py

Environment setup

Target recording time: 05:10 to 06:19

Environment setup was a substantial part of the project. The local setup script targets Ubuntu on an x86-64 computer and recommends Python 3.8 for its legacy RKNN Toolkit2 version. We followed the virtual environment approach so the project dependencies could remain separate from other Python installations.

The documented dependencies include Ultralytics version 8.0.147, NumPy version 1.23.5, and ONNX and ONNX Runtime version 1.10.0. The requirements file also specifies OpenCV for image processing. These are the project's documented versions, rather than a claim that every newer toolkit requires them.

RKNN required a compatible wheel installation in addition to the ordinary dependencies. Before conversion, importing RKNN from rknn dot api is an essential check. The instructions also provide import checks for PyTorch and the ONNX packages. These checks help identify an environment problem before it appears as a conversion failure.

Sources (not spoken): Local setup_yolov8_rknn_ubuntu.sh, requirements.txt, INSTRUCTIONS.md

Model preparation

Target recording time: 06:19 to 07:26

The model preparation script loads the pretrained YOLOv8 nano weights and exports them to ONNX. It requests an input height of four hundred eighty pixels and width of six hundred forty pixels. The export uses operator set twelve, and the repository supports a batch size of one.

These details are part of the interface between the model and the surrounding code. The conversion scripts and inference scripts need to agree on the same dimensions and model path. A valid file with incompatible settings can still fail at the next stage.

The exported ONNX filename includes the height and width, which makes that configuration visible in the model directory. Once the ONNX file is available, the RKNN conversion begins. Exporting and converting are related operations, but they create different artifacts for different tools.

Sources (not spoken): Local pytorch2onnx.py, README.md

Conversion stage 1

Target recording time: 07:26 to 08:34

The first conversion stage uses the script named ONNX to RKNN step one. It collects JPEG image paths from the dataset folder and writes them into a text manifest. The saved manifest contains fifty-one entries. These images support calibration during quantization.

Calibration gives the conversion process sample inputs so it can characterize model behavior. It does not train a new detector, and it does not measure detection accuracy against labeled answers.

The script configures zero mean values and standard deviation values of two hundred fifty-five for each input channel. It sets the target to RK3588, loads the ONNX model, and selects the output node called output zero. It then calls the first hybrid quantization function. The resulting data, model, and quantization configuration files move into the config folder, where the second stage can read them.

Sources (not spoken): Local onnx2rknn_step1.py, dataset.txt, config/

Conversion stage 2

Target recording time: 08:34 to 09:40

The second stage uses ONNX to RKNN step two. This script reads the intermediate files produced by stage one and passes them into the second hybrid quantization function. After a successful build, it exports the RKNN model and moves that file into the model folder.

The saved quantization configuration contains custom layer entries marked float sixteen. That supports describing the process as a hybrid precision workflow. We should not assume that every layer uses exactly the same numerical representation.

The second script then initializes the Toolkit runtime and runs image inference. This is why both script names matter in our explanation. Step one prepares the calibration and conversion intermediates. Step two completes the model export and includes the path that generates detection images. Repeating step one would leave out those later responsibilities.

Sources (not spoken): Local onnx2rknn_step2.py, config/yolov8n-480-640.quantization.cfg

Conversion screen demonstration

Target recording time: 09:40 to 11:46

RECORDING CUE (not spoken): Switch to your Ubuntu recording. Allow 60 seconds for the visible demo, in addition to narration. Use a previously captured run or edit out long waits. Do not rerun conversion over your only saved artifacts without preserving them.

For the demonstration, we will switch to the project terminal and show the environment and conversion sequence. We begin in the project folder and activate the virtual environment. Then we run step one and show the intermediate files in the config directory.

Next, we run step two and show the exported RKNN file in the model directory. The terminal output should connect the conversion command with the generated artifact. We then open the saved detection image from the result directory.

Conversion can take longer than the time allocated in this presentation. For the recording, we can use a captured run and shorten the waiting portion while keeping the commands and completion messages visible. The goal is to show the actual conversion sequence clearly and connect it to the files it produces.

Sources (not spoken): Local conversion scripts and generated artifacts

Image inference and processing

Target recording time: 11:46 to 12:52

The default input image is bus dot JPEG in the dataset folder. The result folder contains the outputs. This directory distinction is important when describing what the scripts actually do.

OpenCV reads the image, and the preprocessing function converts its color ordering from BGR to RGB. Letterboxing fits the photograph into the required input dimensions while preserving its aspect ratio. The gray padding visible in our saved image comes from that preparation.

After inference, the code filters low-confidence predictions using a threshold of zero point two five. Non-maximum suppression uses an intersection-over-union threshold of zero point four five to reduce overlapping duplicate boxes. The visualization function draws the remaining boxes and labels. It saves the prepared image and a side-by-side comparison with the annotations, making the result easy to inspect.

Sources (not spoken): Local onnx2rknn_step2.py and utils.py

Saved object detection result

Target recording time: 12:52 to 13:59

This is the actual saved visualization from our project. The left half shows the prepared image without detection annotations. The right half shows the predictions, with red boxes and green labels. We can see a bus detection and three person detections.

The bus label shows a confidence score of about zero point eight six. Two readable person labels show about zero point eight eight and zero point seven eight. Another label overlaps nearby text, which makes its score harder to read.

These scores belong to individual predictions. They do not mean that the entire system has eighty-six percent accuracy. A proper accuracy evaluation would require a labeled collection of images and a comparison between predictions and ground truth. This example demonstrates a visible detection output and gives us a concrete result to discuss.

Sources (not spoken): Local result/visual_image.jpg

Conversion artifacts and model size

Target recording time: 13:59 to 15:04

The model files provide another measurable result. The saved ONNX model contains approximately thirteen point seven seven million bytes. The RKNN model contains approximately five point three nine million bytes. Comparing those file sizes, the RKNN artifact takes about thirty-nine percent of the ONNX artifact's storage, or approximately sixty-one percent less.

This is useful evidence about the saved files, but we should interpret it carefully. The two formats contain different deployment representations. The size difference does not tell us how much memory inference uses, how much energy the board consumes, or how many frames it can process each second.

Together, the intermediate conversion files and the final RKNN artifact show the outputs of the conversion workflow. A recorded terminal run makes the connection between the commands and these artifacts clearer.

Sources (not spoken): Local model/yolov8n-480-640.onnx and model/yolov8n-480-640.rknn

Group reflections and next experiments

Target recording time: 15:04 to 16:15

Our group learned that applied AI requires coordination across many components. Dependency versions, model paths, tensor dimensions, calibration files, and image processing all influence the final result. Understanding those connections helped us explain the workflow instead of only repeating commands.

The evidence available here supports a functional conversion and detection demonstration. It does not include recorded board execution logs, a labeled accuracy evaluation, or repeated performance measurements. The ONNX and RKNN paths also share output filenames, so the saved image alone does not identify which runtime produced it.

For future work, we would preserve unique outputs for each run and compare both formats on a separate labeled image set. We would then measure repeated inference on the intended RK3588 board, including warm-up runs and separate timing for image processing. That would let us evaluate accuracy and deployment performance more precisely.

Sources (not spoken): Local project artifacts and PROJECT_DISCUSSION.md

References

Target recording time: 16:15 to 17:00

Our main references include the official Ultralytics documentation for YOLOv8, the ONNX introduction, and the ONNX Runtime documentation. Rockchip's Toolkit2 repository explains the conversion and deployment software stack, and the RKNN Model Zoo supplies further examples.

We also acknowledge the upstream detection repository by laitathei, which provides the basis for the scripts used in this project. Our project-specific observations come from the local setup instructions, conversion scripts, saved models, and output image. The complete source links are included in the speaker notes so they remain available with the presentation.

Sources (not spoken): https://docs.ultralytics.com/models/yolov8/
https://onnx.ai/onnx/intro/
https://onnxruntime.ai/docs/
https://github.com/airockchip/rknn-toolkit2
https://github.com/airockchip/rknn_model_zoo
https://github.com/laitathei/YOLOv8-ONNX-RKNN-HORIZON-TensorRT-Detection

Slide 16 / 16

Closing Thoughts

Target recording time: 17:00 to 17:59

To close, our main achievement was completing the practical connection between a pretrained object detector and the RKNN conversion workflow. We had to set up ONNX and RKNN, understand their different purposes, and use both conversion stages to produce the deployment model.

The saved detection image gives us a visible result, while the model artifacts show the outputs of conversion. We also learned to separate those observations from broader claims about accuracy and hardware speed. Those claims need additional experiments.

This project gave our group a clearer understanding of model portability and the responsibilities of preprocessing, inference, and postprocessing. It also gave us a foundation for future work on the intended edge platform. Thank you for watching our presentation.

Sources (not spoken): Local PROJECT_DISCUSSION.md